

Embedding Model & Vector Database Benchmark Report

By Insharah Irfan Nazir

1. Executive Summary.....	1
2. Methodology.....	2
2.1 Corpus.....	2
2.2 Evaluation Queries.....	2
2.3 Embedding Models Tested.....	2
2.4 Vector Databases Tested.....	3
2.5 Metrics.....	4
3. Results: Embedding Models.....	5
3.1 Overall Retrieval Quality.....	5
3.2 Performance by Language.....	6
3.3 Precision@k.....	7
3.4 The Tradeoff: Quality vs Speed.....	7
4. Results: Vector Databases.....	8
4.1 Search Latency.....	8
4.2 Recall vs FAISS Ground Truth.....	9
4.3 Build Time & Memory.....	9
4.4 The Tradeoff: Speed vs Accuracy.....	10
5. Cross-Cut Analysis.....	11
5.1 Model × Database Interaction.....	11
6. Practical Recommendations.....	11
For this corpus (10,456 chunks, 3 languages):.....	11
At larger scale (millions of chunks):.....	12
7. Limitations.....	12
8. Conclusion.....	13

1. Executive Summary

This report evaluates five embedding models and three vector databases for semantic search over a 10,456-chunk corpus spanning English, Korean, and Urdu documents. I measured retrieval quality through recall, precision, and MRR, alongside search latency, build time, and memory usage.

Among the embedding models, BGE-M3 emerges as the strongest performer. It achieves the highest recall@1 at 0.556 and the best MRR at 0.661, maintaining consistent performance across all three languages. On the database side, Chroma delivers the lowest latency at 1.4 milliseconds median with near-perfect recall at 0.989 against the FAISS ground truth. Taken together, the recommended stack for this corpus size is **Chroma** paired with either **BGE-M3** or **multilingual-e5-large**, a combination that balances speed, accuracy, and simplicity.

2. Methodology

2.1 Corpus

The benchmark corpus consists of 63 documents split into 10,456 chunks using recursive character splitting with roughly 50-token overlap. The documents are distributed across three languages: 43 English documents contributing 8,474 chunks, 11 Korean documents with 1,028 chunks, and 9 Urdu documents containing 954 chunks.

Before embedding, I computed each document's alphabetic-character ratio to flag potential extraction noise — garbled OCR output, HTML boilerplate, or encoding artifacts. Ten documents scored notably low, with alpha-ratios ranging from 0.63 to 0.80. These included one OCR-derived PDF (Rubaiyat2_OCR.pdf at 0.63) and several Korean and Urdu HTML sources (0.65–0.80). Rather than excluding these noisy documents, I retained them in the benchmark to reflect real-world conditions, where retrieval systems routinely encounter imperfect source text. This means the reported recall and precision figures include some inherently harder, lower-quality documents, and true model performance on clean text may be modestly higher than the blended numbers shown here.

2.2 Evaluation Queries

I constructed 45 evaluation queries total: 22 in English, 12 in Korean, and 11 in Urdu. Each query has exactly one ground-truth relevant chunk — a held-out sentence drawn from its source document. The Korean queries are near-exact substrings, which makes the retrieval task somewhat easier for that language. For this reason, I recommend comparing models within Korean rather than treating Korean scores as directly comparable to English absolute performance.

A caveat on query distribution: query coverage is uneven at the document level. English queries draw from 11 documents, Korean from 7, but Urdu from only 3. Urdu's strong recall scores should be interpreted with this concentration in mind — with so few source documents, a small number of well-embedded documents can disproportionately drive the language-level average. This is a corpus-design limitation rather than a signal about model quality.

2.3 Embedding Models Tested

Five embedding models were selected to represent different design philosophies and capabilities:

Model	Dims	Context	Description
BGE-large-en	1024	512	English baseline
BGE-M3	1024	8192	Multilingual, long context

E5-large-v2	1024	512	English baseline
multilingual-e5-large	1024	512	Multilingual
Instructor-XL	768	512	Instruction-based

Embedding speed:

Model	Total Time	ms/chunk
BGE-large-en	1,055s	101
BGE-M3	975s	93
E5-large-v2	1,075s	103
multilingual-e5-large	880s	84
Instructor-XL	3,807s	364

A note on compute: initial local benchmarking on CPU measured roughly 2,173 milliseconds per chunk. At that rate, embedding the full corpus across all five models would have taken roughly 19 hours serially. To make full-corpus benchmarking feasible, embedding generation was moved to a GPU-backed environment (Google Colab). The per-model timings reported above reflect GPU throughput, not the CPU baseline. This is disclosed for reproducibility: anyone re-running this benchmark on CPU-only hardware should expect embedding generation, not retrieval, to be the dominant time cost.

2.4 Vector Databases Tested

Three vector databases, representing different architectural approaches, were tested:

Database	Type	Description
FAISS	Library	Exact brute-force baseline

Chroma	Embedded	Lightweight, persistent
Milvus	Server	Distributed (Docker)

A critical design choice: all three were run with exact (FLAT) indexes to ensure fair comparison. At 10,456 vectors, exact search is fast enough that approximate indexes are unnecessary. This removes the recall/latency tradeoff from the database comparison and isolates the pure performance of each system.

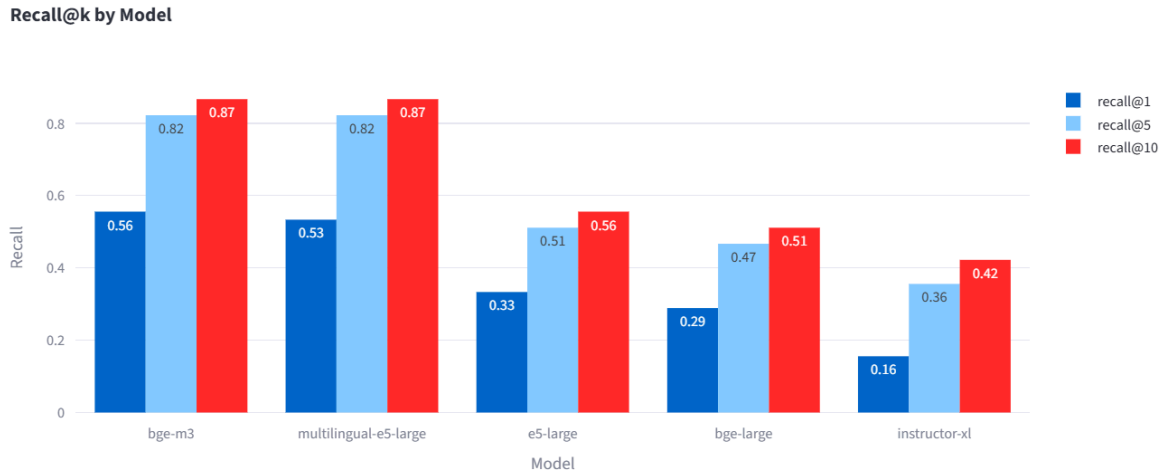
2.5 Metrics

Metric	Definition
Recall@k	Fraction of true top-k nearest neighbors returned
Precision@k	Fraction of returned results that are relevant
MRR	Mean reciprocal rank of the first correct result
Latency	p50, p95, p99 search times
Build time	Time to index all vectors
Memory	RAM footprint of the index

FAISS exhaustive search (IndexFlatIP) establishes the ground truth for all queries. All recall metrics are measured against this exact baseline.

3. Results: Embedding Models

3.1 Overall Retrieval Quality



Model	Recall@1	Recall@5	Recall@10	MRR
bge-m3	0.556	0.822	0.867	0.661
multilingual-e5-large	0.533	0.822	0.867	0.655
e5-large	0.333	0.511	0.556	0.402
bge-large	0.289	0.467	0.511	0.364
instructor-xl	0.156	0.356	0.422	0.243

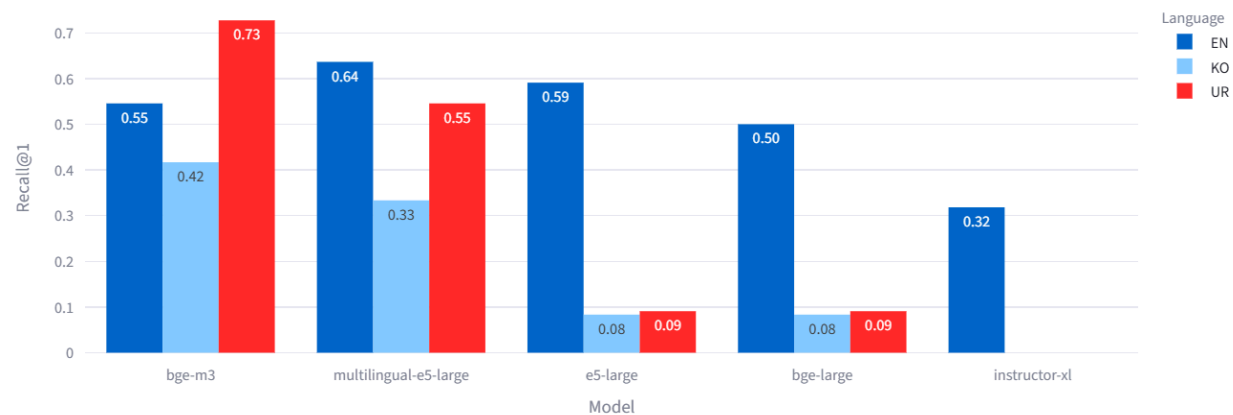
BGE-M3 and multilingual-e5-large stand out as the clear leaders in retrieval quality. Both achieve recall@1 around 0.55 and MRR around 0.66, significantly outperforming the English-only models. The gap between recall@1 and recall@10 is substantial — roughly 0.31 to 0.55 points — which indicates that even when the exact best chunk doesn't land in the top position, the correct answer is often still present somewhere in the top ten results.

For RAG applications, this is an important property: if an LLM processes the top ten chunks, it has roughly an 87% chance of encountering the relevant information, even when the rank ordering is imperfect.

3.2 Performance by Language

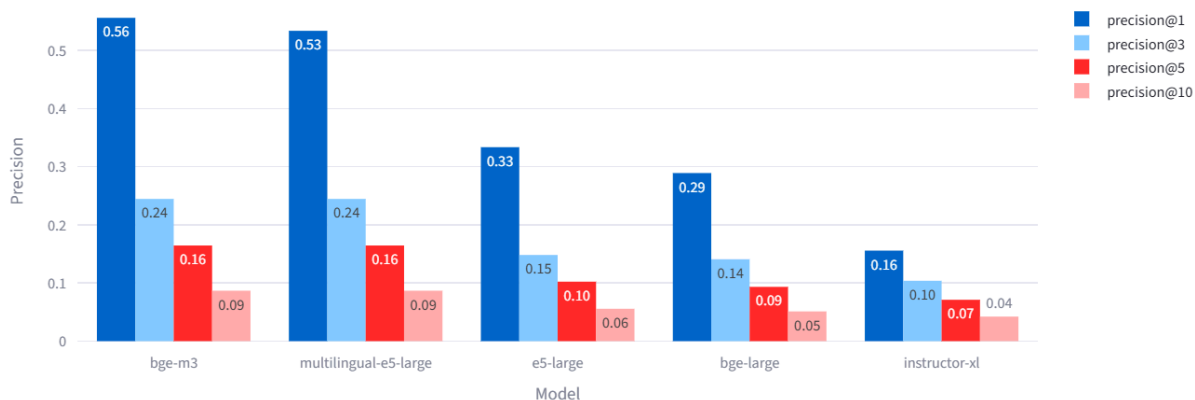
The language breakdown reveals sharp differences in multilingual capability. BGE-M3 maintains consistent performance across all three languages, with its best result actually coming on Urdu at 0.727 recall@1. Multilingual-e5-large also performs well, though with a slight drop on Korean at 0.333. The English-only models — BGE-large and E5-large — demonstrate the expected collapse on non-English text, falling to roughly 8–9% recall. Most strikingly, Instructor-XL fails completely on Korean and Urdu, achieving 0% recall in both languages.

Recall@1 by Model and Language



Model	English Recall@1	Korean Recall@1	Urdu Recall@1
bge-m3	0.545	0.417	0.727
multilingual-e5-large	0.636	0.333	0.545
e5-large	0.591	0.083	0.091
bge-large	0.500	0.083	0.091
instructor-xl	0.318	0.000	0.000

3.3 Precision@k

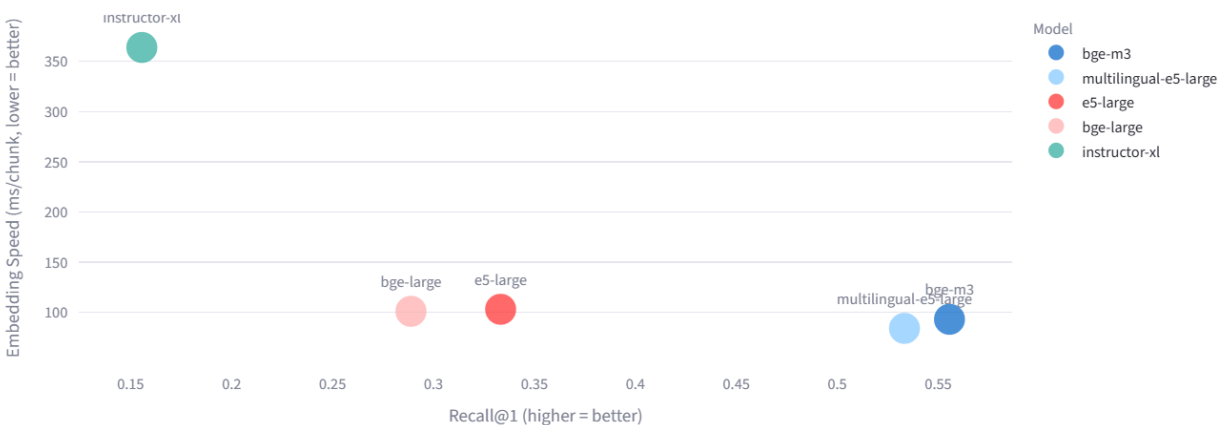


Precision decays steadily as k increases, which is expected given that each query has only a single relevant chunk. Precision@1 is identical to recall@1 by definition. By k=10, precision for the top models has dropped to roughly 0.09, meaning roughly one in every ten retrieved chunks is the ground-truth chunk.

This is expected behavior for single-relevant-document retrieval. In a real RAG system where multiple chunks can be relevant, precision@k would be higher.

3.4 The Tradeoff: Quality vs Speed

Recall@1 vs Embedding Speed (ms per chunk)



Instructor-XL presents a clear case of strict domination: it has the lowest recall and the slowest embedding speed by a wide margin. At 364 milliseconds per chunk, it's roughly four times slower than the other models, while also delivering the worst retrieval quality. There is no scenario where Instructor-XL is the right choice for this task.

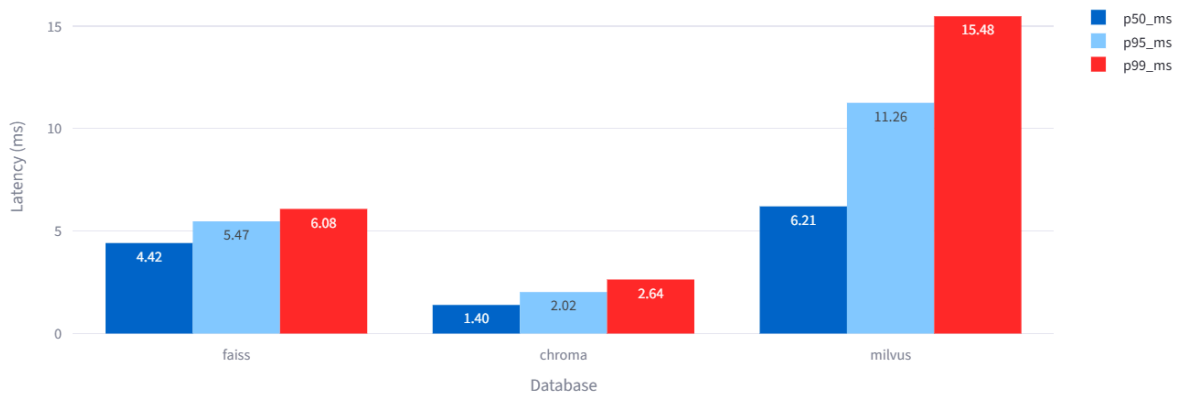
The multilingual models (BGE-M3 and multilingual-e5-large) occupy the sweet spot, offering the best tradeoff between speed and quality.

4. Results: Vector Databases

4.1 Search Latency

Chroma is the fastest database by a substantial margin. Its median latency of 1.4 milliseconds and p99 of 2.6 milliseconds place it well ahead of FAISS at 4.4 milliseconds median and Milvus at 6.2 milliseconds. Milvus also exhibits the heaviest tail latency, with its p99 of 15.5 milliseconds being roughly 2.5 times its median — a level of variability that could matter in production systems with strict latency requirements.

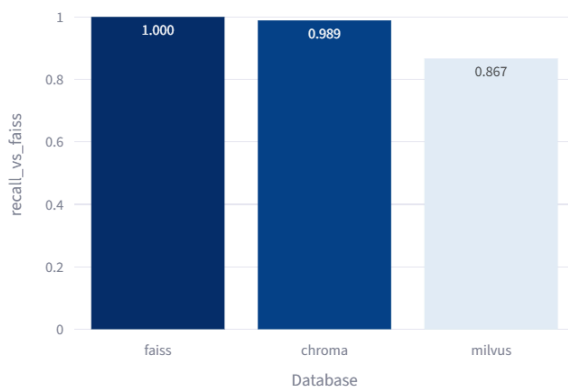
Search Latency Percentiles (ms)



Milvus is slower because, as a server database running in Docker containers, every query incurs TCP round-trip overhead that embedded databases like Chroma and FAISS don't pay. Milvus is designed for distributed scale, not for single-machine performance at 10,000 vectors. This is an architectural tradeoff rather than a performance deficiency.

4.2 Recall vs FAISS Ground Truth

Recall vs FAISS Ground Truth



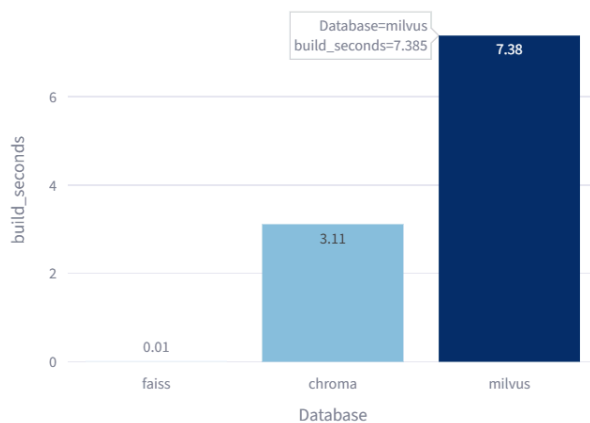
Chroma achieves near-perfect recall at 0.989 while also being the fastest database — an ideal combination. Milvus lags at 0.867, but this gap requires careful interpretation. When both databases use exact (FLAT) indexes, Milvus and FAISS agree almost perfectly on queries where the correct answer stands out clearly from its neighbors.

The disagreement only surfaces in a specific edge case: when the true 10th- and 11th-best matches are separated by a tiny margin — roughly 0.00002 in cosine similarity, essentially a coin flip. In these close calls, Milvus's segment-based distributed search occasionally picks the "wrong" neighbor, while FAISS's single-pass exhaustive search does not.

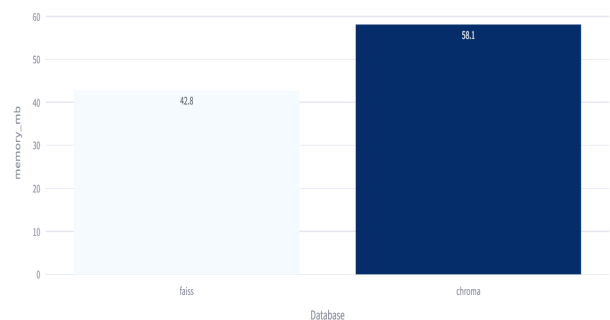
It was verified that this is not a data integrity issue: row counts matched and all inserts succeeded. This is a narrow tie-breaking quirk at the retrieval boundary, not lost or corrupted data.

4.3 Build Time & Memory

Index Build Time (seconds)



Index Memory Usage (MB)



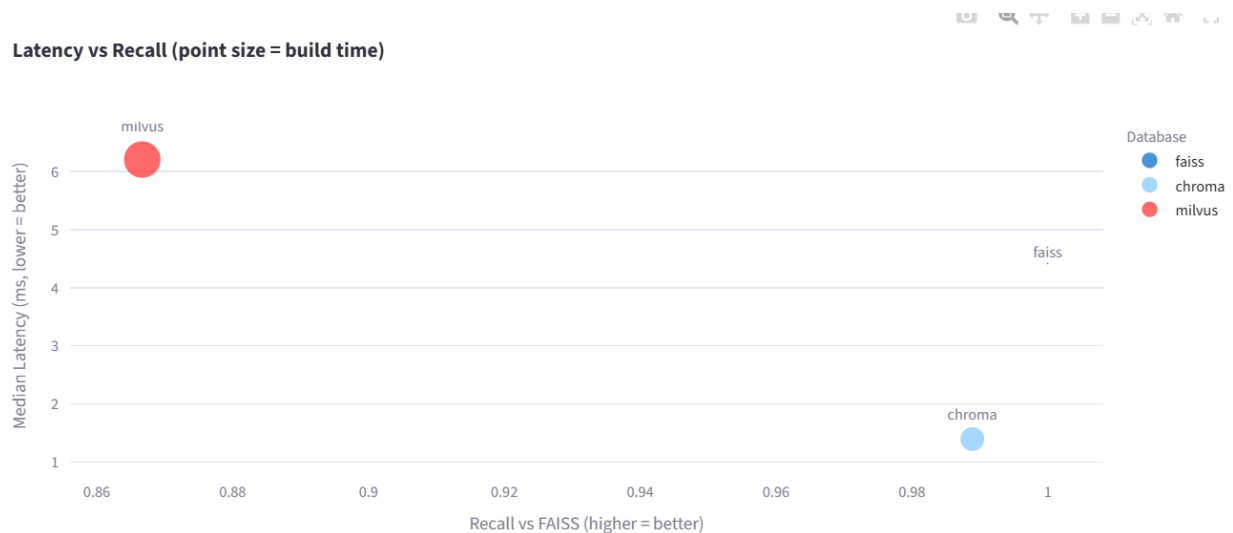
Database	Build Time	Memory
----------	------------	--------

FAISS	0.01s	43 MB
Chroma	3.11s	58 MB
Milvus	7.39s	N/A (out-of-process)

Key observations:

- FAISS builds instantly (0.01s): it's just a library, no persistence.
- Chroma builds in ~3s with moderate memory (~58 MB).
- Milvus is slowest to build due to Docker overhead and segment management.

4.4 The Tradeoff: Speed vs Accuracy



Chroma is the clear sweet spot: fastest query latency, near-perfect recall, and modest memory usage. FAISS is the exact baseline but slower than Chroma. Milvus offers no advantage at this scale — its strengths in horizontal scaling, persistence, and distributed operations only become relevant at much larger scales.

5. Cross-Cut Analysis

5.1 Model × Database Interaction



With exact (FLAT) indexes, retrieval quality is determined by the embedding model, not the database. All databases return the same top-k results modulo minor tie-breaking differences. The heatmap confirms minimal variation across databases for the same model.

This finding is practically useful: it confirms that embedding model and vector database are independent choices. You can select the best model and the best database independently without worrying about unexpected interactions.

6. Practical Recommendations

The strongest overall recommendation is Chroma paired with BGE-M3. This combination delivers the fastest database with the best embedding model, offering low latency, near-perfect recall, and strong multilingual performance. For teams that prioritize embedding speed over marginal gains in recall, Chroma with multilingual-e5-large is a close alternative — the same database with slightly faster embedding (84 vs 93 ms/chunk) at the cost of a modest recall drop. For zero-dependency deployments, FAISS with BGE-M3 provides exact search without any server or persistence overhead, though it's slower than Chroma. The combination of Milvus and Instructor-XL should be avoided entirely — it delivers the worst performance on every metric with no compensating advantage at this scale.

For this corpus (10,456 chunks, 3 languages):

Priority	Recommendation	Rationale
----------	----------------	-----------

Best overall	Chroma + BGE-M3	Fastest DB + best model. Low latency (1.4ms), near-perfect recall, multilingual.
Speed-optimized	Chroma + multilingual-e5-large	Same DB, slightly faster embedding (84 vs 93 ms/chunk). Slightly lower recall.
Zero dependencies	FAISS + BGE-M3	No server, no persistence. Exact search. Slower than Chroma but simpler.
Avoid	Milvus + Instructor-XL	Worst latency, worst recall, worst embedding speed. No advantage at this scale.

At larger scale (millions of chunks):

The recommendations shift dramatically. Milvus is purpose-built for distributed scaling and would become competitive at larger volumes. Approximate indexes like HNSW and IVF would become necessary for any system, fundamentally changing the recall/latency tradeoff. Chroma and FAISS are single-machine solutions that do not scale horizontally.

7. Limitations

1. Single-machine benchmarking: Latency numbers are from a laptop with Docker running. Production server numbers would differ.
2. Single relevant chunk per query: Precision@k is artificially low because each query has only 1 correct answer. In real RAG, multiple chunks can be relevant.
3. Korean queries are near-exact: Korean test queries are held-out sentences from the corpus, not natural questions. This makes the task easier (substring matching) and should not be compared to English absolute scores.
4. Exact indexes only: I did not test approximate indexes (HNSW, IVF). At 10k vectors, exact is the right choice. At 1M vectors, the tradeoff story would be different.
5. Docker overhead: Milvus latency includes containerization overhead that would be lower in a bare-metal production deployment.

8. Conclusion

For a 10,456-chunk multilingual corpus, Chroma with BGE-M3 provides the best combination of speed (1.4ms p50), accuracy (0.556 recall@1, 0.989 vs FAISS), and simplicity (embedded, no server). FAISS remains a strong zero-dependency baseline, and Milvus is overkill at this scale.

The embedding model choice matters more than the database choice — BGE-M3 and multilingual-e5-large are the top performers, while Instructor-XL is strictly dominated (slowest and worst).